UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas · Escuela de Ingeniería de Software

GUÍA PRÁCTICA — SEMANA 10

Sprint 1 e integración continua con GitHub Actions

SW705 — Taller de Ingeniería de Software I

Docente: Mg. Ing. José Carlos, García La Riva

Caso aplicado: Clínica San Rafael S.A.C.

Integrantes del equipo	Albino Soto, Christopher Henrry · Inga Champi, Franz Joe · Rashuaman Sapallanay, Ricco Didier
Código(s)	20231158C · 20231302G · 20231242D
Sección	SW705-U · Grupo A
Fecha	9 de noviembre de 2026
Repositorio del equipo	https://github.com/francito69/SW705U-grupo-A

Estado de esta entrega (corte: 9 de noviembre de 2026)

Esta versión completa la planificación del Sprint 1: asignación, Definición de Hecho, archivo ci.yml, criterios de aceptación, regla de protección prevista y estado del Sprint.
A la fecha, el repositorio no contiene código de la aplicación ni flujo de GitHub Actions. Por eso, los pasos que requieren ejecución (partes 2 a 7) y sus evidencias se indican como PENDIENTE. Los únicos resultados informados son los obtenidos realmente; no se incluyen capturas ni salidas simuladas.

Instrucciones generales

En esta práctica el equipo avanza el Sprint 1 de su proyecto integrador: construye una historia con su prueba automática, activa la integración continua con GitHub Actions, protege la rama principal y deja registrado el incremento. Los ejemplos usan el caso Clínica San Rafael S.A.C. (FastAPI, pytest); si su proyecto usa otra tecnología, adapte los comandos del flujo y conserve el mismo orden de trabajo.

Modalidad y evaluación

Aspecto	Detalle
Modalidad	En equipo, con trabajo individual dentro del equipo: cada integrante construye su historia en su propia rama y el equipo comparte el flujo de integración continua.
Duración	4 horas de taller (Bloques: 0.5 h Daily y tablero, 1.5 h historia, 1 h integración continua, 1 h pull request, protección y registro).
Requisitos	Repositorio de la Semana 9 con README, .gitignore y rama principal; Python 3.12 o superior; Git; cuenta de GitHub de cada integrante.
Evaluación	Formativa: el docente retroalimenta con la rúbrica del final de este documento. No promedia en las prácticas: la calificación sumativa del Avance 5 es en la Semana 11.
Entregable	Repositorio con el flujo de CI activo, pull request de la historia con el check en verde, rama principal protegida y este documento completado con evidencias.

Reglas del trabajo

Ninguna historia llega a la rama principal sin pull request, sin el check de CI en verde y sin la aprobación de otro integrante.

Nunca se suben datos reales de pacientes, contraseñas ni claves: solo datos ficticios en pruebas y ejemplos.

Si el build se pone rojo, se lee el registro antes de pedir ayuda; el mensaje de error casi siempre indica el archivo y la línea.

Cada evidencia debe poder verificarse en el repositorio (enlace a la ejecución, al pull request o al commit).

Parte 1 — La jornada de rodaje: tablero y Definición de Hecho

Antes de escribir código, el equipo revisa el tablero del Sprint 1 y confirma quién hace qué hoy. Luego actualiza la Definición de Hecho (DoD) acordada en la Semana 7 con una condición nueva: el build de integración continua en verde.

1.1 Asignación de la jornada

Integrante	Historia o tarea de hoy	Puntos	Rama	Estado al iniciar
Inga Champi, Franz Joe	HU03 Registro y búsqueda de paciente: estructura base del backend y endpoints GET y POST /pacientes	3	`feature/HU03-registro-paciente`	Por hacer
Rashuaman Sapallanay, Ricco Didier	Tarea de HU03: pipeline de GitHub Actions y pruebas del documento duplicado (409)	(en HU03)	`ci/github-actions`	Por hacer
Rashuaman Sapallanay, Ricco Didier	HU02 Agendamiento de citas sin solapamientos: POST /citas con 409 ante un cruce	5	`feature/HU02-agendamiento-citas`	Por hacer
Albino Soto, Christopher Henrry	HU01b Acceso por rol y bitácora: pantalla de inicio de sesión y menú según el rol	3	`feature/HU01b-acceso-rol`	Por hacer

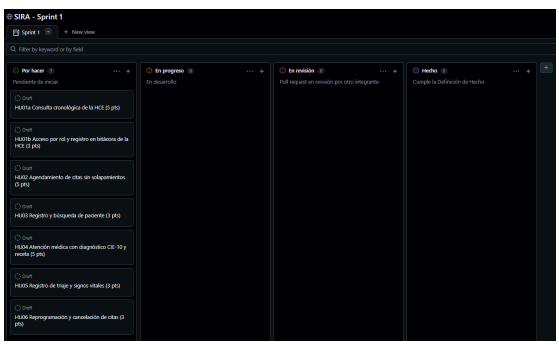
Cada fila debe corresponder a una historia del Sprint Backlog de la Semana 7 y a una rama con nombre según la convención de la Semana 9.

1.2 Definición de Hecho con integración continua

N.º	Criterio verificable de la Definición de Hecho	¿Cómo se verifica?
1	El código está integrado en main mediante un pull request aprobado por un integrante distinto del autor.	Pull request aprobado en GitHub (pestaña Reviews).
2	Las pruebas de la historia pasan y la cobertura de la lógica del módulo es de 80 % o más.	pytest --cov=app --cov-fail-under=80 (backend); Vitest (frontend).
3	Los escenarios Dado/Cuando/Entonces de la historia están verificados.	Una prueba automática por escenario o demostración al Product Owner.
4	NUEVO — El build de integración continua está en verde: el check test (ruff y pytest con cobertura) pasa antes de fusionar.	Check test en verde en el pull request y enlace a la ejecución en Actions.
5	Cada endpoint nuevo está documentado con sus códigos de respuesta.	Swagger de FastAPI en /docs.
6	Las operaciones sobre datos clínicos validan el rol (403), quedan en bitacora_auditoria y no usan borrado físico.	Prueba con un rol no autorizado y registro en la bitácora.
7	No hay errores abiertos de prioridad alta y el Product Owner aceptó la historia en la Sprint Review.	Tablero sin errores de prioridad alta; tarjeta en Hecho con aprobación.

Evidencia: captura del tablero con las historias del Sprint 1 y de la tabla anterior completada.

Estado: la tabla está completada. La captura disponible es la del tablero registrado en la Semana 7; captura actualizada: PENDIENTE.



Tablero «SIRA - Sprint 1» (Entregable 5, Semana 7): 7 historias, 27 puntos.

Parte 2 — Punto de partida: ejecutar el proyecto en su equipo

Antes de automatizar, cada integrante comprueba que el proyecto funciona en su computadora. Lo que no corre en su equipo no puede correr en el servidor de integración.

N.º	Qué hacer	Comando o acción

2.1	Clonar el repositorio del equipo (o traer los últimos cambios si ya lo tiene) y entrar a la carpeta.	`git clone https://github.com/USUARIO/REPO.git`
		`cd REPO`
		`git switch main`
		`git pull`

2.2	Crear y activar un entorno virtual. En Windows se activa con venv\Scripts\activate.	`python3 -m venv venv` `source venv/bin/activate`
2.3	Instalar las dependencias del proyecto.	`pip install -r requirements.txt`
2.4	Ejecutar las pruebas existentes y anotar cuántas pasan.	`pytest -q`
2.5	Levantar la aplicación y abrir la documentación automática en el navegador (detener con Ctrl + C).	`uvicorn app.main:app --reload` Abrir http://127.0.0.1:8000/docs

Dato	Resultado en su equipo
Versiones (equipo de Inga Champi)	Python 3.12.10 · git version 2.46.2.windows.1
Pruebas existentes (pytest -q)	no tests ran in 0.06s (código de salida 5): 0 pruebas; el repositorio no tiene carpeta tests/.
Aplicación y página /docs	*No ejecutable: el repositorio no tiene requirements.txt ni carpeta app/.*

Evidencia: captura de pytest -q con el resultado y de la página /docs.

Estado: resultados reales del 9/11/2026. Pasos 2.2, 2.3 y 2.5 y captura de /docs: PENDIENTE hasta que exista el backend.

Parte 3 — Activar la integración continua con GitHub Actions

La tarea de esta parte es de equipo: la ejecuta una persona (en la solución, Ana, mantenedora del repositorio) y las demás la revisan. El flujo se escribe en un archivo del repositorio y GitHub lo ejecuta en cada push y en cada pull request hacia la rama principal.

Qué hace cada línea del flujo

on: define cuándo se ejecuta (push a main y pull requests hacia main).

jobs / runs-on: define un trabajo llamado test que corre en una máquina Ubuntu de GitHub.

steps: pasos en orden: descargar el código, preparar Python, instalar dependencias, revisar el estilo y ejecutar las pruebas. Si un paso falla, los siguientes no se ejecutan y el check queda en rojo.

Complete los espacios _____ del archivo de la Parte 3.3 usando esta explicación.

N.º	Qué hacer	Comando o acción

3.1	Partir de la rama principal actualizada y crear la rama de trabajo.	`git switch main`
		`git pull`
		`git switch -c ci/github-actions`

3.2	Crear el archivo requirements-dev.txt con las herramientas de desarrollo (no se instalan en producción).	`ruff==0.17.4` `pytest-cov==7.1.0`
3.3	Crear la carpeta .github/workflows y dentro el archivo ci.yml con el contenido del cuadro de la página siguiente.	`mkdir -p .github/workflows`

3.4	Registrar el cambio y subir la rama. No ejecute ruff en su equipo todavía: queremos ver qué detecta el servidor.	`git add .github requirements-dev.txt`
		`git commit -m "ci: agrega flujo de GitHub Actions con ruff y pytest"`
		`git push -u origin ci/github-actions`

3.5	En GitHub, abrir un pull request de ci/github-actions hacia main y abrir la pestaña Checks (o Actions) para ver la ejecución.	GitHub → Pull requests → New pull request

Archivo .github/workflows/ci.yml

```
name: CI

on:
  push:
    branches: [ main   ]          # ¿en qué rama?
  pull_request:
    branches: [ main   ]

jobs:
  test  :                         # nombre del job (será el check requerido)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12  "
          cache: pip
      - name: Instalar dependencias
        run: pip install   -r requirements.txt -r requirements-dev.txt
      - name: Revisar estilo con ruff
        run: ruff check --select E4,E7,E9,F app tests
      - name: Ejecutar pruebas con cobertura
        run: pytest --cov=app --cov-fail-under=80
```

Si su proyecto no usa Python, adapte la instalación y las pruebas a su tecnología conservando la estructura on / jobs / steps.

Evidencia: enlace al pull request del flujo y captura de la pestaña Checks mientras se ejecuta.

Estado: archivo resuelto. Aún no está en .github/workflows/ porque el flujo requiere las carpetas app/ y tests/. Pasos 3.1 a 3.5, enlace al pull request y captura de Checks: PENDIENTE.

Parte 4 — El primer build rojo y el primer build verde

Lo normal es que la primera ejecución falle. En el código de referencia de la clínica, ruff detecta un import que no se usa en app/routers/citas.py. Aprender a leer un build rojo es el objetivo de esta parte. Si el build de su proyecto salió verde de inmediato, provoque el rojo agregando la línea import os (sin usarla) al inicio de app/main.py.

N.º	Qué hacer	Comando o acción
4.1	Abrir la ejecución fallida en la pestaña Actions, desplegar el paso que falló y copiar el mensaje de error.	GitHub → Actions → ejecución con → paso "Revisar estilo con ruff"
4.2	Instalar las herramientas y reproducir el error en su equipo con el mismo comando del flujo.	`pip install -r requirements-dev.txt` `ruff check --select E4,E7,E9,F app tests`
4.3	Corregir el problema. En la clínica: quitar models del import de app/routers/citas.py, que queda así:	`from .. import crud, schemas`
4.4	Comprobar localmente que ruff y las pruebas pasan, con los mismos comandos del flujo.	`ruff check --select E4,E7,E9,F app tests` `pytest --cov=app --cov-fail-under=80`

4.5	Registrar la corrección y subirla a la misma rama: el pull request se vuelve a verificar solo.	`git add -A`
		`git commit -m "fix: elimina import sin uso en citas.py"`
		`git push`

4.6	Confirmar en el pull request que el check cambió a verde.	GitHub → pull request → marca verde en Checks

Registro de builds

N.º	Commit	Resultado	Causa del resultado	Corrección aplicada
—	Sin commit de flujo	*No ejecutado*	La API de GitHub informa 0 flujos y 0 ejecuciones (9/11/2026).	*Pendiente: crear el flujo.*
—	—	*Pendiente*	—	—
—	—	*Pendiente*	—	—

Evidencia: captura del registro del build rojo con el error y del build verde final.

Estado: PENDIENTE. Sin ejecuciones no hay build rojo ni verde que documentar; no se registran resultados supuestos.

Parte 5 — Fusionar el flujo y proteger la rama principal

Con el build en verde, otro integrante revisa el pull request del flujo y lo fusiona. Después la mantenedora activa la protección: la rama principal solo aceptará cambios con pull request, una aprobación y el check de pruebas en verde.

N.º	Qué hacer	Comando o acción
5.1	El revisor abre la pestaña Files changed, revisa el archivo del flujo y aprueba el pull request. Luego se fusiona (Squash and merge) y se borra la rama.	GitHub → pull request → Review changes → Approve → Squash and merge

5.2	Actualizar la copia local.	`git switch main`
		`git pull`
		`git branch -d ci/github-actions`

5.3	Crear la regla de protección de la rama principal.	GitHub → Settings → Rules → Rulesets → New ruleset → New branch ruleset
5.4	Nombrar la regla, activarla (Enforcement: Active) y apuntar a la rama principal.	Target branches → Add target → Include default branch
5.5	Marcar: exigir pull request antes de fusionar (1 aprobación) y exigir que los checks pasen, agregando el check test. Bloquear los push forzados.	Require a pull request before merging Require status checks to pass → Add checks → test Block force pushes

5.6	Probar la protección: intentar un push directo a main y leer el rechazo.	`git switch main`
		`git commit --allow-empty -m "prueba: push directo"`
		`git push`
		`git reset --hard HEAD~1`

Si su plan de GitHub no permite reglas en repositorios privados

Las reglas de protección en repositorios privados requieren un plan de pago; en repositorios públicos están disponibles. Con el visto bueno del docente, puede hacer el repositorio público siempre que contenga solo datos ficticios y ningún secreto.

Si no es posible, documente el acuerdo de equipo ("no se fusiona sin check verde y sin aprobación") y adjunte una captura de un pull request que lo cumplió.

El check test solo aparece en la lista después de que el flujo se haya ejecutado al menos una vez.

Configuración de la regla	Valor elegido por el equipo
Nombre de la regla	proteccion-main
Aplicación y rama objetivo	Enforcement: Active · Include default branch (main)
Pull request antes de fusionar	Obligatorio, con 1 aprobación de otro integrante
Checks de estado requeridos	test (disponible cuando el flujo se haya ejecutado una vez)
Push forzados	Bloqueados (Block force pushes)

Evidencia: captura de la regla configurada y del rechazo del push directo (o el acuerdo de equipo, si aplica la nota anterior).

Estado: valores previstos para la regla; aún no creada (el repositorio tiene 0 reglas) porque el check test no existe. Pasos 5.1 a 5.6 y capturas: PENDIENTE.

Parte 6 — Una historia con la prueba escrita primero

Cada integrante repite esta parte con su propia historia. El ejemplo de la clínica es HU-04 Cancelar o reprogramar cita. Primero se traducen los criterios de aceptación a pruebas; las pruebas fallan; después se escribe el código que las hace pasar.

6.1 De los criterios de aceptación a las pruebas

Criterio de aceptación (historia propia)	Prueba automática	Resultado esperado
HU03 E1: con documento nuevo y campos obligatorios, el sistema crea el paciente.	`test_registra_paciente_nuevo_con_convenio`	201 y paciente disponible en la búsqueda por documento.
HU03 E1: el sistema valida el formato del correo y del teléfono.	`test_rechaza_correo_o_telefono_invalido`	422 y el paciente no se crea.
HU03 E2: no se permite el mismo tipo y número de documento.	`test_no_permite_documento_duplicado`	409 «Paciente ya registrado» con el id del paciente existente.

N.º	Qué hacer	Comando o acción

6.2	Partir de main actualizada (ya con el flujo de CI) y crear la rama de la historia.	`git switch main`
		`git pull`
		`git switch -c feature/HU-04-cancelar-cita`

6.3	Escribir las pruebas en tests/test_citas.py (ver cuadro) y ejecutarlas: debe fallar la del criterio nuevo.	`pytest -q`

6.4	Registrar solo las pruebas, subir la rama y abrir el pull request: el check debe ponerse rojo por la prueba que falla.	`git add tests/test_citas.py`
		`git commit -m "test: cubre cancelacion de citas (HU-04)"`
		`git push -u origin feature/HU-04-cancelar-cita`

6.5	Escribir el código mínimo que hace pasar las pruebas (ver cuadro) y ejecutar pruebas y ruff.	`pytest -q` `ruff check --select E4,E7,E9,F app tests`

6.6	Registrar el código y subirlo: el pull request se vuelve a verificar y debe quedar en verde.	`git add app`
		`git commit -m "feat: impide cancelar citas atendidas (HU-04)"`
		`git push`

6.7	Otro integrante revisa con la lista de verificación de la Semana 9, aprueba y se fusiona (Squash and merge). Borrar la rama.	GitHub → Review changes → Approve → Squash and merge

6.8	Actualizar la copia local y limpiar la rama.	`git switch main`
		`git pull`
		`git branch -d feature/HU-04-cancelar-cita`

Código de las pruebas (tests/test_citas.py)

Escriba aquí, en su repositorio, las pruebas de los criterios de su historia. Use como modelo las pruebas existentes de tests/ (por ejemplo, test_no_permite_citas_duplicadas_mismo_horario). Una prueba por criterio de aceptación.

Estado: criterios tomados del ERS (HU03). Las pruebas aún no están escritas: el repositorio no tiene tests/. Pasos 6.2 a 6.8: PENDIENTE.

Resultados de ejecución

Momento	Comando	Resultado
Estado actual del repositorio (9/11/2026)	`pytest -q`	no tests ran in 0.06s (código de salida 5)
Solo con las pruebas (rojo)	`pytest -q`	*Pendiente*
Con el código mínimo (verde)	`pytest -q`	*Pendiente*
Revisión de estilo	`ruff check --select E4,E7,E9,F app tests`	*Pendiente*

Lista de verificación del revisor

Pregunta de revisión	Sí / No	Comentario
¿Hay una prueba automática por cada criterio de aceptación?	*Pend.*	Sin pull request de la historia que revisar.
¿El historial muestra rojo antes del código y verde después?	*Pend.*	Sin pull request de la historia que revisar.
¿El check test está en verde y la cobertura es de 80 % o más?	*Pend.*	Sin pull request de la historia que revisar.
¿Los commits y la rama siguen la convención del equipo?	*Pend.*	Sin pull request de la historia que revisar.
¿Los endpoints están documentados y validan el rol (403)?	*Pend.*	Sin pull request de la historia que revisar.
¿Solo hay datos ficticios y ningún secreto en el cambio?	*Pend.*	Sin pull request de la historia que revisar.

Evidencia: enlace al pull request con el historial de checks (rojo y luego verde) y la aprobación del revisor.

Estado: PENDIENTE. La lista se aplicará cuando otro integrante revise el pull request de HU03.

Parte 7 — Cobertura e insignia del build

Un build verde solo es tan confiable como las pruebas que lo respaldan. El flujo ya exige una cobertura mínima de 80 %. Ahora el equipo revisa qué partes del código no tienen pruebas y deja a la vista el estado del build en el README.

N.º	Qué hacer	Comando o acción
7.1	Ejecutar las pruebas mostrando las líneas no cubiertas por módulo.	`pytest --cov=app --cov-report=term-missing`
7.2	Identificar el módulo con menor cobertura y proponer una prueba que lo mejore.	Anotar en la tabla siguiente
7.3	Agregar al inicio del README la insignia del flujo (reemplace USUARIO y REPO).	`![CI](https://github.com/USUARIO/REPO/actions/workflows/ci.yml/badge.svg)`

7.4	Subir el cambio con un pull request pequeño de documentación, siguiendo el mismo proceso.	`git switch -c docs/insignia-ci`
		`git add README.md`
		`git commit -m "docs: agrega insignia del build al README"`
		`git push -u origin docs/insignia-ci`

Módulo	Cobertura	Líneas sin cubrir	Prueba propuesta
app/ (no existe aún)	*Sin datos*	—	Primera prueba prevista: documento duplicado de HU03 (409).
—	—	—	—
—	—	—	—

Las cifras corresponden al código de referencia de la clínica antes de HU-04; las de su proyecto serán distintas.

Buenas prácticas del flujo

Los secretos (claves de API, contraseñas de base de datos) se guardan en Settings → Secrets del repositorio y nunca en el código ni en el archivo del flujo.

Fije las versiones de las dependencias (requirements.txt y requirements-dev.txt) para que el build de hoy sea igual al de mañana.

Para dar solo los permisos necesarios al flujo, puede agregar al archivo, antes de jobs:, el bloque permissions: con contents: read.

Evidencia: captura del informe de cobertura y de la insignia visible en el README.

Estado: PENDIENTE. Sin código no hay informe de cobertura. La insignia no se agrega aún porque apuntaría a un flujo inexistente.

Parte 8 — Registro del incremento y retrospectiva breve

Al cerrar la jornada, el equipo deja constancia del estado real del Sprint 1: qué historias están hechas según la DoD, cuánto trabajo falta y qué cambia para la Semana 11.

8.1 Estado del Sprint Backlog

Historia	Puntos	Estado	Evidencia (pull request)
HU01a Consulta de la HCE y HU01b Acceso por rol y bitácora	5 + 3	Por hacer	Sin pull request
HU02 Agendamiento de citas sin solapamientos	5	Por hacer	Sin pull request
HU03 Registro y búsqueda de paciente	3	Por hacer	Sin pull request
HU04 Atención médica con CIE-10 y receta	5	Por hacer	Sin pull request
HU05 Registro de triaje y signos vitales	3	Por hacer	Sin pull request
HU06 Reprogramación y cancelación de citas	3	Por hacer	Sin pull request

8.2 Avance del Sprint (puntos pendientes)

Corte	Línea ideal	Puntos pendientes reales	Observación
Inicio del Sprint (Semana 9)	27	27	Sprint Backlog de este equipo: 27 puntos (la guía usa 24 como referencia).
Fin de la Semana 9	18	27	Ninguna historia cumple la Definición de Hecho.
Semana 10 (corte 9/11/2026)	9	27	Sigue sin haber código: desvío de 18 puntos.
Fin de la Semana 11	0	*Por registrar*	Riesgo alto: el Sprint Backlog debe revisarse con el equipo.

Línea ideal: 24 puntos que bajan en partes iguales hasta 0 al final de la Semana 11. Los puntos pendientes reales se calculan sumando las historias que aún no cumplen la Definición de Hecho.

8.3 Retrospectiva de la jornada

Qué funcionó	Qué mejorar	Acción concreta para la Semana 11 (responsable)
Sprint Backlog, Definición de Hecho y tablero definidos desde la Semana 7.	El Sprint 1 inició y aún no hay código en el repositorio.	Inga Champi: publicar la estructura base del backend y HU03 por pull request antes de la Semana 11.
Esquema SQL de 22 tablas y datos ficticios listos.	La integración continua todavía no está activa.	Rashuaman Sapallanay: activar ci.yml y luego la regla de protección antes de la Semana 11.
Documentación de las entregas disponible en el repositorio.	Hubo cambios directos a main, sin revisión.	Albino Soto (Scrum Master): actualizar el tablero y exigir pull request en todo cambio.

Evidencia: capturas del tablero y del gráfico o tabla de avance, y de la retrospectiva.

Estado: tabla de avance y retrospectiva completadas. Captura actualizada del tablero: PENDIENTE.

Anexo — Problemas frecuentes con la integración continua

Síntoma	Causa probable	Qué hacer
El flujo no se ejecuta al subir cambios.	El archivo no está en .github/workflows/ o no termina en .yml; o la rama del push no coincide con la de on:.	Verificar la ruta exacta y los nombres de rama; abrir la pestaña Actions.
Error de sintaxis del flujo (Invalid workflow file).	Sangría incorrecta en el YAML: se usan espacios, nunca tabuladores.	Revisar la línea que indica el mensaje y alinear con 2 espacios por nivel.
Pasa en mi computadora y falla en el servidor.	Falta una dependencia en requirements.txt o requirements-dev.txt, o la versión de Python es distinta.	Instalar en un entorno virtual nuevo con los archivos de requisitos y probar de nuevo.
ModuleNotFoundError al ejecutar pytest.	Dependencia sin declarar o error de importación.	Agregarla al archivo de requisitos con versión fija.
ruff reporta F401 (import sin uso) o F821 (nombre no definido).	Se quitó o se olvidó un import.	Ejecutar el mismo comando de ruff en su equipo y corregir la línea indicada.
Falla por cobertura (Required test coverage of 80% not reached).	Las pruebas no cubren suficiente código.	Ver las líneas sin cubrir con --cov-report=term-missing y agregar pruebas.
El check test no aparece al crear la regla de protección.	El flujo todavía no se ejecutó, o el job tiene otro nombre.	Ejecutar el flujo una vez y usar el nombre exacto del job.
Authentication failed al hacer push.	Token vencido o sin permisos.	Generar un token de acceso personal con permiso sobre el repositorio o usar SSH.
No se pueden crear reglas de protección en el repositorio.	Repositorio privado en un plan que no incluye la función.	Ver la nota de la Parte 5.

Rúbrica de evaluación — Semana 10

Escala vigesimal (0–20). La guía es formativa: la rúbrica orienta la retroalimentación del docente y prepara la calificación sumativa del Avance 5 de la Semana 11. Subir datos reales de pacientes o secretos al repositorio se califica como nivel Inicial en el criterio correspondiente.

Criterio (peso)	Logrado (18–20)	Satisfactorio (14–17)	En proceso (11–13)	Inicial (0–10)
Tablero, asignación y Definición de Hecho (15 %)	Cada integrante tiene una historia real del Sprint Backlog con rama convencional; la DoD incluye el build verde y todos los criterios son verificables.	Asignación completa y DoD con CI; algún criterio es poco verificable.	Asignación o DoD incompletas; la DoD no incorpora la integración continua.	No presenta asignación ni DoD.
Flujo de integración continua (25 %)	Flujo activo en el repositorio con instalación, ruff y pruebas con cobertura mínima; se ejecuta en push y pull request hacia main; el equipo explica cada sección del archivo.	Flujo activo con instalación y pruebas; falta la revisión de estilo o la cobertura mínima; explicación parcial.	Archivo del flujo presente pero con ejecuciones fallidas sin resolver o ejecuta solo parte del proceso.	No hay flujo de integración continua.
Historia con la prueba primero (25 %)	Cada criterio de aceptación tiene una prueba automática; el historial del pull request muestra rojo antes del código y verde después; el código es mínimo y legible.	Pruebas para los criterios principales; el historial de rojo a verde no es del todo claro.	Pruebas que no corresponden a los criterios, o escritas después sin evidenciar la falla inicial.	No hay pruebas automáticas de la historia.
Pull request, revisión y protección (15 %)	Pull request revisado y aprobado por otro integrante con la lista de verificación, fusionado con el check en verde; rama principal protegida (o acuerdo documentado si el plan no lo permite).	Revisión y fusión correctas; la protección está configurada de forma parcial.	Se fusiona sin revisión de otro integrante o con el check en rojo; sin protección ni acuerdo.	Cambios directos a la rama principal.
Diagnóstico del build y cobertura (10 %)	Registro de builds con causa y corrección verificables; identifica el módulo menos cubierto y propone una prueba concreta; la insignia está en el README.	Registro con causa y corrección; análisis de cobertura parcial.	Registro sin causa clara o sin análisis de cobertura.	No documenta los builds.
Registro del incremento y reflexión (10 %)	Estado del Sprint coherente con la DoD, avance calculado correctamente y retrospectiva con una acción concreta, con responsable y plazo.	Estado y avance correctos; la retrospectiva es general.	Estado inconsistente con la DoD o avance mal calculado.	No presenta registro.

Guía de verificación para el docente

Dónde se encuentra la evidencia de cada criterio. El repositorio puede revisarse desde el enlace, sin clonarlo.

Dónde verificar	Qué buscar
Documento y tablero del equipo (criterio 1)	Asignación coherente con el Sprint Backlog; DoD con la condición del build verde.
Repositorio → .github/workflows/ci.yml y pestaña Actions (criterio 2)	Archivo con on, jobs y pasos de instalación, estilo y pruebas; ejecuciones recientes; al menos un rojo y un verde.
Pull request de la historia → Checks y commits (criterio 3)	El primer commit con solo pruebas queda en rojo; el siguiente con el código queda en verde.
Pull request → Reviews y Settings → Rules (criterio 4)	Aprobación de un integrante distinto del autor; regla con el check test requerido.
Registro de builds, informe de cobertura y README (criterio 5)	Causas coherentes con los registros de Actions; insignia visible; módulo menos cubierto identificado.
Tablero, avance y retrospectiva (criterio 6)	Puntos pendientes coherentes con las historias que cumplen la DoD.

Calificación

Criterio	Peso	Nivel alcanzado	Nota (0–20)	Nota ponderada
Tablero, asignación y Definición de Hecho	15 %			
Flujo de integración continua	25 %			
Historia con la prueba primero	25 %			
Pull request, revisión y protección	15 %			
Diagnóstico del build y cobertura	10 %			
Registro del incremento y reflexión	10 %			
NOTA FORMATIVA DE LA SEMANA	100 %			